

DATA ANALYTICS PORTFOLIO PROJECT

FIFA 21 Player Dataset

Data Cleaning & Exploratory Data Analysis (EDA)

Prepared by: BRAMWEL SAMWEL MWASENGA

Role: Data Analyst

DATE : 09, 2026

1. Project Title & Overview

Title: FIFA 21 Player Dataset — Data Cleaning & Exploratory Data Analysis (EDA)

A full exploratory data analysis and cleaning project on the raw FIFA 21 player dataset (18,979 players, 77 columns). The dataset arrived heavily "dirty" — corrupted characters, mixed-unit text fields, and numeric values stored as text — and required systematic diagnosis and cleaning before it could be used for any real analysis.

Domain: Sports Analytics / Data Cleaning / Exploratory Data Analysis

2. Business Problem / Question

- What problem are you solving? Raw player data exported from a sports database was not analysis-ready: currency values, physical measurements, and ratings were all stored as inconsistent text, with corrupted character encoding throughout the file.
- Why does this matter? Clubs, analysts, and game designers rely on this data to compare player value, performance stats, and market worth. Dirty data leads to broken calculations, failed sorting, and incorrect insights.
- What decisions could this analysis inform? Player market valuation comparisons, scouting shortlists by physical/performance attributes, and squad-building budget planning.

3. Dataset Information

- Source of data: FIFA 21 player ratings dataset (publicly available, originally scraped from SoFIFA.com).
- Number of rows and columns: 18,979 rows × 77 columns (88 after cleaning, including new derived columns).
- Time period covered: FIFA 21 season player ratings (single snapshot).
- Key variables/columns: Name, Age, Nationality, OVA (Overall Rating), Value, Wage, Release Clause, Height, Weight, Team & Contract, plus 50+ individual skill attributes.

4. Tools & Technologies Used

- Software: Python, Microsoft Excel
- Libraries: pandas (data manipulation), numpy (numeric handling), matplotlib (visualization), re (regex text parsing), openpyxl (Excel I/O)
- Version: Python 3.11, pandas 2.x

Excel vs. Python — Division of Work

Both tools were used deliberately, each for the task it handles best:

Task	Excel	Python
Initial data inspection & structure review (rows/columns, headers)	✓	—
Spot-checking raw values & manual formula checks (COUNTBLANK, sort/filter)	✓	—

Encoding corruption fixes (€, ★, é)	—	✓
Currency parsing: text → numeric (K/M suffixes)	—	✓
Height/Weight unit conversion	—	✓
Splitting combined text fields	—	✓
Duplicate detection cross-check	✓	✓
Missing-value flagging & boolean columns	✓	✓
Outlier review & manual verification	✓	—
Final data type conversion & validation	—	✓
Before/After comparison visuals	—	✓

Excel was used for fast, visual inspection of the raw file — scanning headers, spot-checking values, sorting/filtering to catch outliers, and manually verifying flagged rows. Python (pandas) was used for the actual cleaning logic — anything repeatable, rule-based, or requiring text parsing across thousands of rows, since it is faster, auditable, and reproducible at scale.

5. Data Cleaning & Preparation

Eleven distinct data-quality issues were identified and resolved:

- 1. Column Header Encoding** — Renamed the column to a clean 'OVA'.
- 2. Character Encoding Corruption (Mojibake)** — Applied character-mapping replacement to restore correct UTF-8 symbols.
- 3. Currency Columns: Text → Numeric** — Parsed into clean numeric columns (Value_EUR, Wage_EUR, ReleaseClause_EUR), converting K/M suffixes to full numeric values.
- 4. Height/Weight: Mixed-Unit Text → Numeric** — Converted to numeric Height_cm and Weight_kg columns.
- 5. Team & Contract: Split Multi-line Field** — Split into two clean columns: Team and Contract_Years.
- 6. Star Ratings: Symbol Text → Numeric** — Extracted the numeric rating only into new integer columns (W/F_num, SM_num, IR_num).
- 7. Hits Column: Strip Whitespace → Numeric** — Stripped whitespace and converted to a numeric (integer) column.
- 8. Loan Date End: Sparse Column → Boolean Flag** — Converted to a boolean On_Loan flag instead of imputing fake dates.
- 9. Duplicate Record Removal** — Removed the duplicate using drop_duplicates().
- 10. Age Outlier Review (Max Age = 53)** — Manually reviewed and confirmed as a legitimate veteran/legend player record; retained in the dataset.
- 11. Overall Data Types: Text → Numeric** — Converted all relevant columns to proper numeric dtypes (float64/int64).

6. Analysis & Methodology

- Techniques used: Missing-value profiling, duplicate detection, data-type auditing, regex-based text parsing, unit conversion, and boolean flag engineering.

Example code snippet (currency parsing):

```
def money_to_num(x):
    if pd.isna(x): return np.nan
    x = str(x).replace('€', '').strip()
    if x.endswith('M'):
        return float(x[:-1]) * 1_000_000
    elif x.endswith('K'):
        return float(x[:-1]) * 1_000
    return float(x)

df['Value_EUR'] = df['Value'].apply(money_to_num)
```

- Statistical methods applied: Descriptive statistics (min/max/mean for Age, Value, Wage), frequency counts for categorical fields, and outlier boundary review for Age.

7. Key Insights & Findings

- 100% of currency and physical-measurement fields (Value, Wage, Release Clause, Height, Weight) were stored as unusable text before cleaning.
- Character encoding corruption affected 7 of 77 columns — nearly 10% of fields displayed incorrect symbols (€, ★, é).
- Only 1 exact duplicate row existed out of 18,979 — the dataset was otherwise well de-duplicated.
- Loan Date End was populated in only 5.3% of records (1,013 players), confirming loan status is best modeled as a boolean flag.
- The oldest player is 53 years old — verified as a legitimate legend-tier player, not a data-entry error.

8. Visualizations — Before / After Screenshots (11 Stages)

Each visual below shows the raw ("Before") value next to the cleaned ("After") value for real records from the dataset, generated directly from the cleaning script.

Stage 1: Column Header Encoding

Issue: The '↓OVA' column header was corrupted to 'â†“OVA' due to a text-encoding error.

Fix applied: Renamed the column to a clean 'OVA'.

Tool used: Excel (manual header inspection)

Stage 1: Column Header Encoding

BEFORE

Column Name (raw)
'ât"OVA'
'POT'
'Age'

AFTER

Column Name (fixed)
'OVA'
'POT'
'Age'

Stage 2: Character Encoding Corruption (Mojibake)

Issue: Euro signs (€), star symbols (★), and accented letters (é) were garbled into sequences like 'â,¬', 'â~...', 'Ã©' across Value, Wage, Release Clause, W/F, SM, IR, and Team & Contract.

Fix applied: Applied character-mapping replacement to restore correct UTF-8 symbols.

Tool used: Python (pandas, str.replace)

Stage 2: Character Encoding Corruption (Mojibake)

BEFORE

Value	Wage	Team & Contract
â,¬67.5M	â,¬560K	/ / / FC Barcelona / 2004 ~ 2021 / /
â,¬46M	â,¬220K	/ / / Juventus / 2018 ~ 2022 / /
â,¬75M	â,¬125K	/ / / AtlÃ©tico Madrid / 2014 ~ 2023 / /
â,¬87M	â,¬370K	/ / / Manchester City / 2015 ~ 2023 / /

AFTER

Value	Wage	Team & Contract (team part)
€67.5M	€560K	FC Barcelona
€46M	€220K	Juventus
€75M	€125K	Atlético Madrid
€87M	€370K	Manchester City

Stage 3: Currency Columns: Text → Numeric

Issue: Value, Wage, and Release Clause were stored as text like "€67.5M", preventing sorting or math operations.

Fix applied: Parsed into clean numeric columns (Value_EUR, Wage_EUR, ReleaseClause_EUR), converting K/M suffixes to full numeric values.

Tool used: Python (pandas, regex)

Stage 3: Currency Columns: Text -> Numeric**BEFORE**

Name	Value	Wage	Release Clause
L. Messi	â,~67.5M	â,~560K	â,~138.4M
Cristiano Ronaldo	â,~46M	â,~220K	â,~75.9M
J. Oblak	â,~75M	â,~125K	â,~159.4M
K. De Bruyne	â,~87M	â,~370K	â,~161M

AFTER

Name	Value_EUR	Wage_EUR	ReleaseClause_EUR
L. Messi	67500000.0	560000.0	138400000.0
Cristiano Ronaldo	46000000.0	220000.0	75900000.0
J. Oblak	75000000.0	125000.0	159400000.0
K. De Bruyne	87000000.0	370000.0	161000000.0

Stage 4: Height/Weight: Mixed-Unit Text → Numeric

Issue: Height (5'7") and Weight (159lbs) were stored as unusable text strings mixing numbers and units.

Fix applied: Converted to numeric Height_cm and Weight_kg columns.

Tool used: Python (pandas, regex)

Stage 4: Height/Weight: Mixed-Unit Text -> Numeric (cm/kg)**BEFORE**

Name	Height	Weight
L. Messi	5'7"	159lbs
Cristiano Ronaldo	6'2"	183lbs
J. Oblak	6'2"	192lbs
K. De Bruyne	5'11"	154lbs

AFTER

Name	Height_cm	Weight_kg
L. Messi	170.2	72.1
Cristiano Ronaldo	188.0	83.0
J. Oblak	188.0	87.1
K. De Bruyne	180.3	69.9

Stage 5: Team & Contract: Split Multi-line Field

Issue: Team & Contract combined team name and contract years in a single field with embedded newline characters.

Fix applied: Split into two clean columns: Team and Contract_Years.

Tool used: Python (pandas, str.split)

D	E	F	G	H	I	J	K	L	M	N	O	P	Q
Nationality	Positions	Name	Age	â€œOVA	POT	Team & Contract	ID	Height	Weight	foot	BOV	BP	C
Argentina	RW ST CF	L. Messi	33	93	93	FC Barcelona 2004 ~ 2021	158023	5'7"	159lbs	Left	93	RW	

Stage 5: Team & Contract: Split Multi-line Field

BEFORE

Name	Team & Contract
L. Messi	\n\nFC Barcelona\n2004 ~ 2021\n\n
Cristiano Ronaldo	\n\nJuventus\n2018 ~ 2022\n\n
J. Oblak	\n\nAtlético Madrid\n2014 ~ 2023\n\n
K. De Bruyne	\n\nManchester City\n2015 ~ 2023\n\n

AFTER

Name	Team	Contract_Years
L. Messi	FC Barcelona	2004 ~ 2021
Cristiano Ronaldo	Juventus	2018 ~ 2022
J. Oblak	Atlético Madrid	2014 ~ 2023
K. De Bruyne	Manchester City	2015 ~ 2023

Stage 6: Star Ratings: Symbol Text → Numeric

Issue: W/F, SM, and IR ratings were stored as text with symbols, e.g. "4 ★", inconsistent in spacing.

Fix applied: Extracted the numeric rating only into new integer columns (W/F_num, SM_num, IR_num).

Tool used: Python (regex)

Stage 6: Star Ratings: Symbol Text -> Numeric

BEFORE

Name	W/F	SM	IR
L. Messi	4 ★...	4★...	5 ★...
Cristiano Ronaldo	4 ★...	5★...	5 ★...
J. Oblak	3 ★...	1★...	3 ★...
K. De Bruyne	5 ★...	4★...	4 ★...

AFTER

Name	W/F_num	SM_num	IR_num
L. Messi	4	4	5
Cristiano Ronaldo	4	5	5
J. Oblak	3	1	3
K. De Bruyne	5	4	4

Stage 7: Hits Column: Strip Whitespace → Numeric

Issue: The Hits column contained leading newline/whitespace characters (e.g. "\n372") and was stored as text.

Fix applied: Stripped whitespace and converted to a numeric (integer) column.

Tool used: Python (pandas)

Stage 7: Hits Column: Strip Whitespace -> Numeric

BEFORE

Name	Hits
L. Messi	\n372'
Cristiano Ronaldo	\n344'
J. Oblak	\n86'
K. De Bruyne	\n163'

AFTER

Name	Hits
L. Messi	372.0
Cristiano Ronaldo	344.0
J. Oblak	86.0
K. De Bruyne	163.0

Stage 8: Loan Date End: Sparse Column → Boolean Flag

Issue: Loan Date End was missing in 94.7% of rows (17,966 of 18,979) — expected, since most players are not on loan.

Fix applied: Converted to a boolean On_Loan flag instead of imputing fake dates.

Tool used: Excel (COUNTBLANK check) + Python (pandas)

Stage 8: Loan Date End: Sparse Column -> Boolean Flag

BEFORE

Name	Loan Date End
L. Messi	NaT
Cristiano Ronaldo	NaT
J. Oblak	NaT
K. De Bruyne	NaT

AFTER

Name	On_Loan
L. Messi	False
Cristiano Ronaldo	False
J. Oblak	False
K. De Bruyne	False

Stage 9: Duplicate Record Removal

Issue: One fully duplicated row was found (Player ID 251698, K. Berlaso).

Fix applied: Removed the duplicate using drop_duplicates().

Tool used: Excel (Conditional Formatting scan) + Python (pandas)

Stage 9: Duplicate Record Removal**BEFORE**

ID	Name	Age
251698	K. Berlaso	32
251698	K. Berlaso	32

AFTER

ID	Name	Age
251698	K. Berlaso	32

Stage 10: Age Outlier Review (Max Age = 53)

Issue: The oldest player in the dataset was 53 years old — flagged as a possible data-entry error.

Fix applied: Manually reviewed and confirmed as a legitimate veteran/legend player record; retained in the dataset.

Tool used: **Excel (Sort & Filter)**

Stage 10: Age Outlier Review (Max Age = 53)**BEFORE**

Name	Age	Flag
K. Miura	53	
H. Sulaimani	43	
L. ButrÃ³n	43	
Hilton	42	

AFTER

Name	Age	Flag
K. Miura	53	Reviewed: legend/veteran player - kept
H. Sulaimani	43	Reviewed: legend/veteran player - kept
L. ButrÃ³n	43	Reviewed: legend/veteran player - kept
Hilton	42	Reviewed: legend/veteran player - kept

Stage 11: Overall Data Types: Text → Numeric

Issue: Multiple numeric-looking columns (Value, Wage, Hits, star ratings) were stored as text (object dtype), blocking correlation analysis and aggregation.

Fix applied: Converted all relevant columns to proper numeric dtypes (float64/int64).

Tool used: **Python (pandas)**

Stage 11: Overall Data Types: Text -> Numeric**BEFORE**

Column	Dtype (before)
Value	object (str)
Wage	object (str)
Hits	object (str)
W/F	object (str)

AFTER

Column	Dtype (after)
Value_EUR	float64
Wage_EUR	float64
Hits	float64
W/F_num	int64

9. Recommendations

- Always validate file encoding (UTF-8) immediately after import from web-scraped sources — mojibake is easy to miss on a quick spot-check.
- Standardize all currency and measurement fields into single-unit numeric columns before any modeling or aggregation.
- Convert sparse date columns into boolean flags rather than imputing fake dates, to avoid introducing false signal.
- Treat outliers as a review task, not an automatic deletion rule — verify against domain knowledge before removing rows.

10. Conclusion

This project took a raw, heavily corrupted 18,979-row FIFA 21 export and turned it into a clean, analysis-ready dataset by systematically diagnosing and fixing 11 distinct data-quality issues — from encoding corruption to mixed-unit text fields. The process demonstrates a complete, real-world data cleaning workflow: profiling, diagnosis, targeted fixes, and verification.

What I learned: How to detect and repair mojibake/encoding corruption, parse mixed-format currency and measurement text with regex, and make defensible decisions about missing values and outliers rather than applying blanket rules.

Potential next steps: Build player market-value prediction models using the cleaned numeric fields, or create an interactive dashboard comparing players by position, nationality, and value-for-rating.